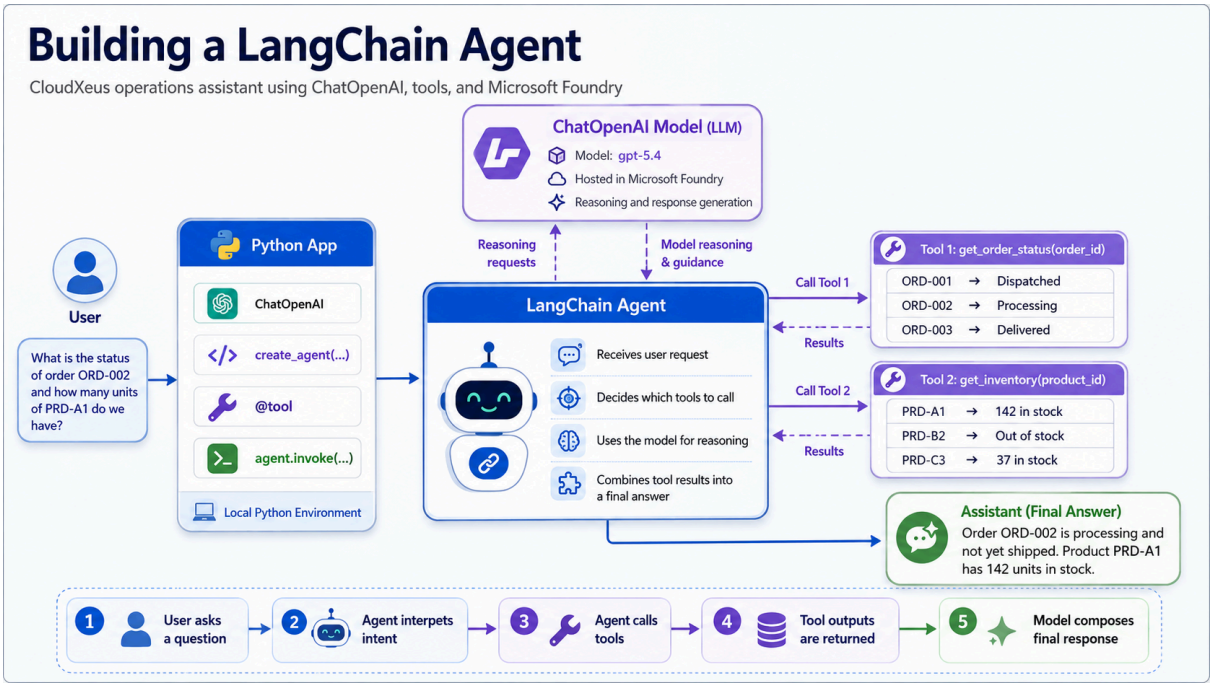
The Wider Ecosystem LangChain, LangGraph, and the Model Context Protocol

Lab - Getting started with LangChain

LangChain - https://www.langchain.com/

Building a LangChain agent



In this chapter, we are looking at how a LangChain agent works with a model, tools, and our local Python application.

The flow starts with the user asking a business question. In this example, the user asks about an order status and also asks how many units of a product are available. This is important because the question cannot be answered by the language model alone. The model does not automatically know the current status of an order or the latest inventory count. That information has to come from tools or business systems.

On the left side, we have the Python application. This is where we define the agent setup. The application uses `ChatOpenAI` to connect to the deployed model in Microsoft Foundry. It also defines tools using the `@tool` decorator. These tools represent actions the agent can perform, such as checking order status or checking product inventory. Finally, the application calls `agent.invoke(...)` to send the user request into the agent.

The LangChain agent sits in the middle of the process. Its job is to interpret the user's request, decide what information is needed, and determine which tools should be called. This is what makes it different from a simple model call. A normal model call only generates a response from the prompt. An agent can reason over the request, choose tools, call them, inspect the results, and then combine everything into a final answer.

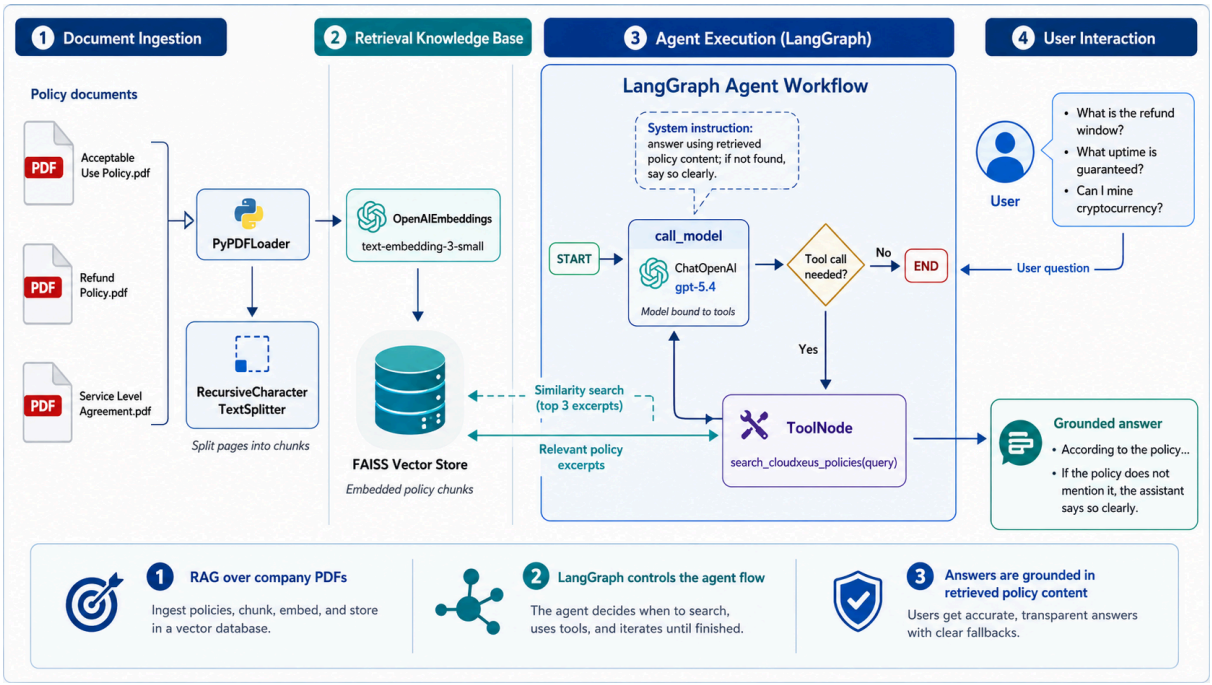
At the top, we have the ChatOpenAI model. In this example, the model is hosted in Microsoft Foundry. The agent uses the model for reasoning and response generation. The model helps the agent understand the user's intent and decide the next step, but the actual live business data comes from the tools.

On the right side, we have the tools. One tool checks the order status based on an order ID. Another tool checks inventory based on a product ID. When the agent decides it needs this information, it calls the appropriate tool and receives the result. For example, the order tool may return that order ORD-002 is processing, while the inventory tool may return that product PRD-A1 has 142 units in stock.

The final answer is then composed by the assistant. The agent does not simply dump the raw tool outputs back to the user. Instead, it uses the model to turn those results into a clear business response, such

as: “Order ORD-002 is processing and has not yet shipped. Product PRD-A1 has 142 units in stock.”
The key takeaway is that a LangChain agent allows us to combine model reasoning with real business actions. The model provides language understanding and reasoning, while tools provide access to live operational data. This pattern is very useful for building AI assistants that need to answer questions using real systems rather than relying only on the model’s general knowledge.

Building an agent using LangGraph - Implementation overview



In this chapter, we are looking at how to build an agent using LangGraph, and how the different pieces fit together in a RAG-based workflow.

Here, we start with company policy documents, such as acceptable use policies, refund policies, and service level agreements. These documents contain the trusted business knowledge that the agent should use when answering questions. Instead of relying only on the model's general knowledge, we first prepare a knowledge base from approved company documents.

The documents are loaded using a PDF loader. The loader reads the PDF files and extracts the text from them. Once the text is extracted, it is split into smaller chunks using a text splitter. This step is important because large documents are usually too big to send to the model in one request. By splitting the documents into smaller sections, we make it easier to search and retrieve only the most relevant content later.

The second part is the retrieval knowledge base. Each chunk of text is converted into an embedding using an embedding model. An embedding is a numerical representation of the meaning of the text. These embeddings are then stored in a vector store, such as FAISS. The vector store allows us to perform similarity search. So when a user asks a question, we can find the policy chunks that are most relevant to that question.

The third part is the LangGraph agent workflow. This is where LangGraph controls the execution flow of the agent. The workflow starts by sending the user's question to the model. The model has system instructions that tell it to answer using retrieved policy content and to say clearly if the answer is not found in the policy.

The important part here is the decision point. The model can decide whether a tool call is needed. If the question requires policy information, the workflow routes the request to a ToolNode. The ToolNode runs the search tool, which queries the vector store and retrieves the most relevant policy excerpts. Those excerpts are then sent back into the agent workflow so the model can use them as grounding context.

If no tool call is needed, or once the tool results have been returned, the workflow moves toward the final answer. This loop is what makes LangGraph useful. It gives us a structured way to control how the

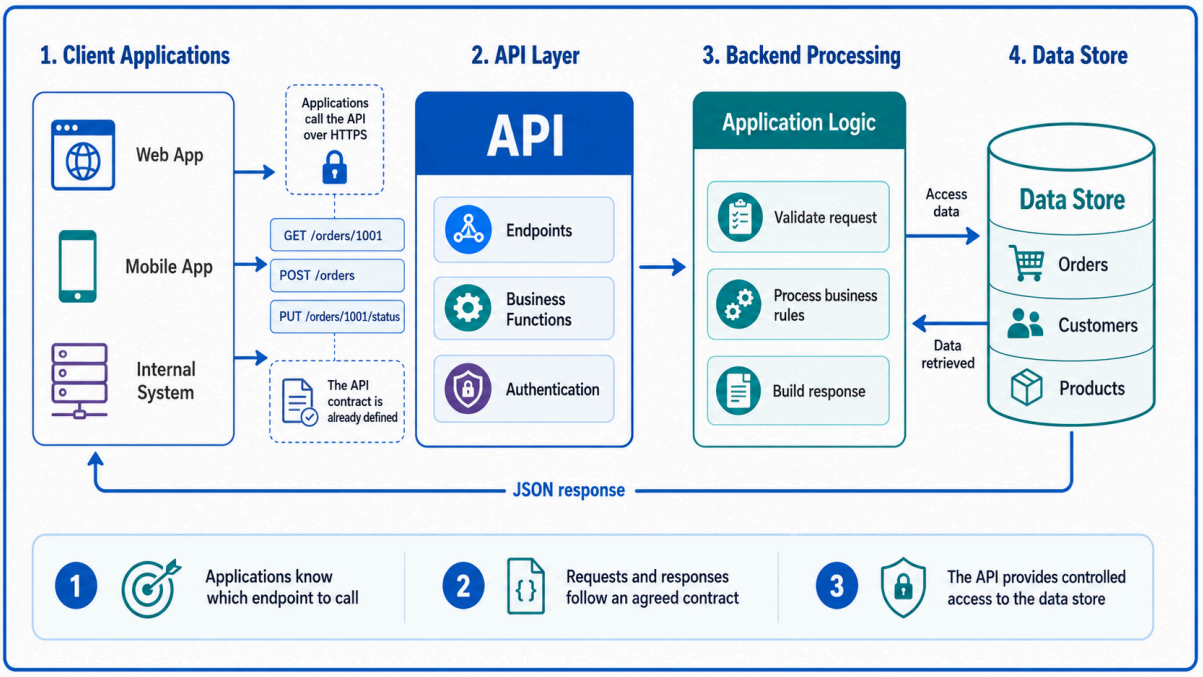
agent reasons, when it calls tools, and when the workflow should end.

From the user's point of view, they simply ask questions such as "What is the refund window?", "What uptime is guaranteed?", or "Can I mine cryptocurrency?" Behind the scenes, the agent searches the policy knowledge base, retrieves the relevant excerpts, and then generates a grounded answer.

The final answer is grounded because it is based on retrieved policy content, not just the model's general training. This is especially important in business scenarios where answers must follow approved company rules. If the policy does not mention something, the agent should not invent an answer. It should clearly say that the policy does not provide that information.

The key takeaway is that LangGraph helps us build a controlled agent workflow. Document ingestion prepares the knowledge base, embeddings and the vector store make retrieval possible, the LangGraph workflow controls the agent's reasoning and tool use, and the final response is grounded in company documents. This pattern is useful for policy assistants, HR helpdesks, IT support agents, compliance tools, and any scenario where users need reliable answers from trusted internal content.

Why was the Model Context Protocol designed

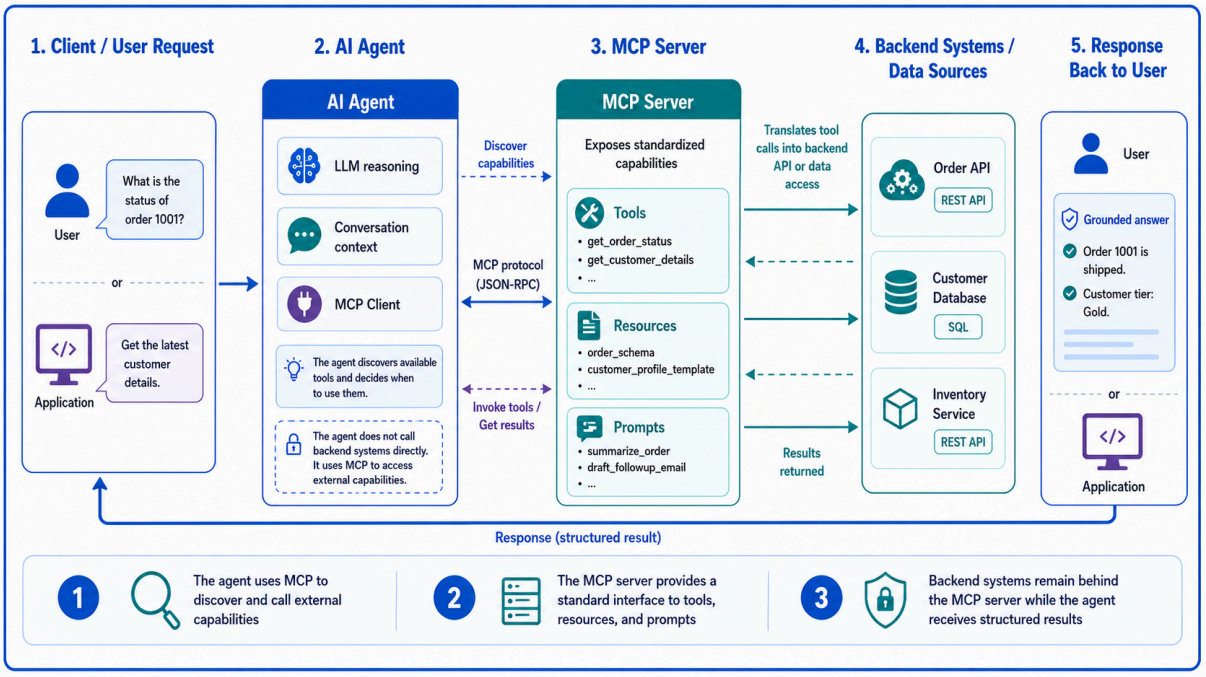


In the traditional API model, applications such as web apps, mobile apps, or internal systems call an API over HTTPS. The application already knows which endpoint to call. For example, it may call an endpoint to get order details, update an order status, or retrieve customer information.

The API layer exposes the endpoints, handles authentication, and defines the contract for requests and responses. This contract is usually fixed in advance. The application developer knows the endpoint URL, the request format, the parameters, and the expected JSON response.

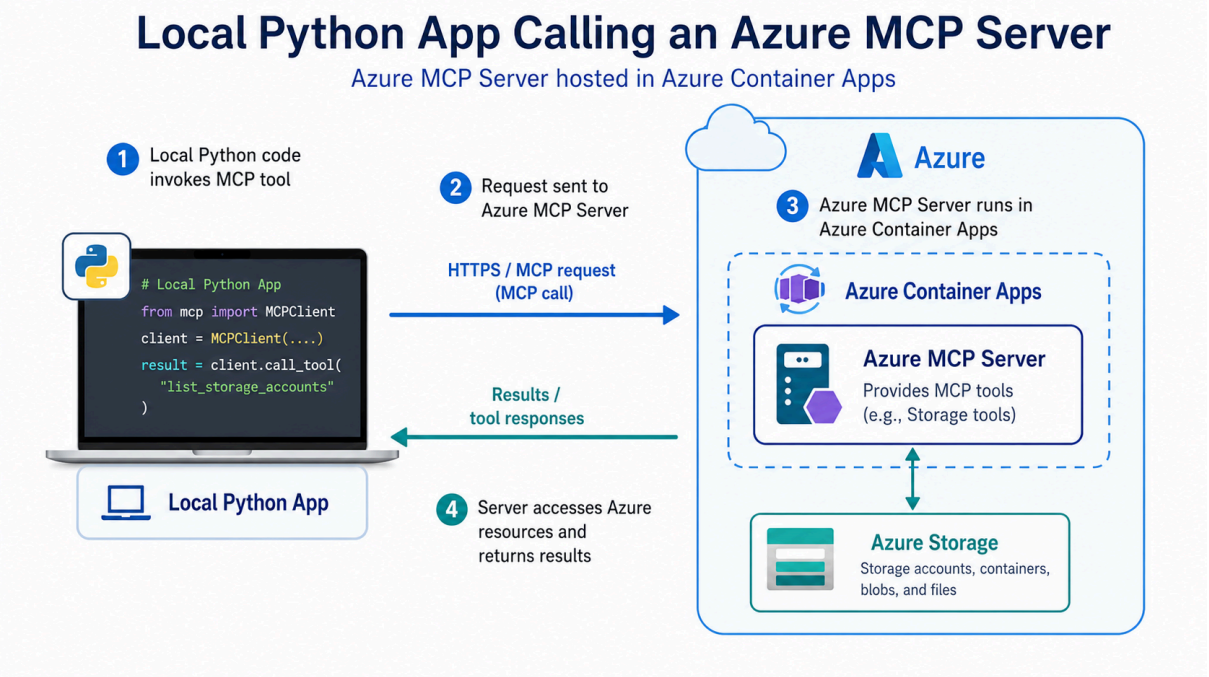
Behind the API, the backend application validates the request, applies business rules, accesses the database, and builds the response. The data store remains protected behind the API, and clients do not connect to the database directly.

So the traditional API approach works very well when the client application is deterministic. In other words, the application is programmed ahead of time to call a specific endpoint for a specific task.


1. Client / User Request
2. AI Agent
3. MCP Server
4. Backend Systems / Data Sources
5. Response Back to User
User
What is the status of order 1001?
or
Application
Get the latest customer details.
AI Agent
LLM reasoning
Conversation context
MCP Client
The agent discovers available tools and decides when to use them.
The agent does not call backend systems directly. It uses MCP to access external capabilities.
Discover capabilities
MCP protocol (JSON-RPC)
Invoke tools / Get results
MCP Server
Exposes standardized capabilities
Tools
• get_order_status
• get_customer_details
• ...
Resources
• order_schema
• customer_profile_template
• ...
Prompts
• summarize_order
• draft_followup_email
• ...
Translates tool calls into backend API or data access
Results returned
Order API
REST API
Customer Database
SQL
Inventory Service
REST API
User
Grounded answer
Order 1001 is shipped.
Customer tier: Gold.
or
Application
Response (structured result)
1
The agent uses MCP to discover and call external capabilities
2
The MCP server provides a standard interface to tools, resources, and prompts
3
Backend systems remain behind the MCP server while the agent receives structured results

But AI agents work differently.
An agent may receive a natural-language request such as, “What is the status of order 1001?” or “Get the latest customer details.” The agent has to reason about the user’s intent, discover what capabilities are available, decide which tool to use, pass the right input, and then use the result to answer the user.
This is where MCP comes in.
Instead of the agent directly knowing every backend API and every contract in advance, the agent connects to an MCP server. The MCP server exposes capabilities in a standardized way. These capabilities can include tools, resources, and prompts.
Tools represent actions the agent can perform, such as `get_order_status` or `get_customer_details`. Resources represent data or context the agent can access, such as schemas, documents, or templates. Prompts can provide reusable instructions or task patterns that help the agent work consistently.
The agent uses an MCP client to communicate with the MCP server. Through the MCP protocol, the agent can discover available capabilities, understand how to call them, invoke the required tool, and receive structured results.
The backend systems still remain protected behind the MCP server. The agent does not directly connect to the database or call every backend system on its own. The MCP server acts as the controlled bridge between the agent and the organization’s systems.
This design is important because it gives agents a more standard way to work with external tools and data sources. Instead of building a custom integration for every agent and every backend system, MCP provides a common pattern for exposing capabilities to AI agents.
So the key difference is this: in the traditional API approach, the application already knows which endpoint to call. In the MCP approach, the AI agent can discover available capabilities through the MCP server and decide which ones to use based on the user’s request.

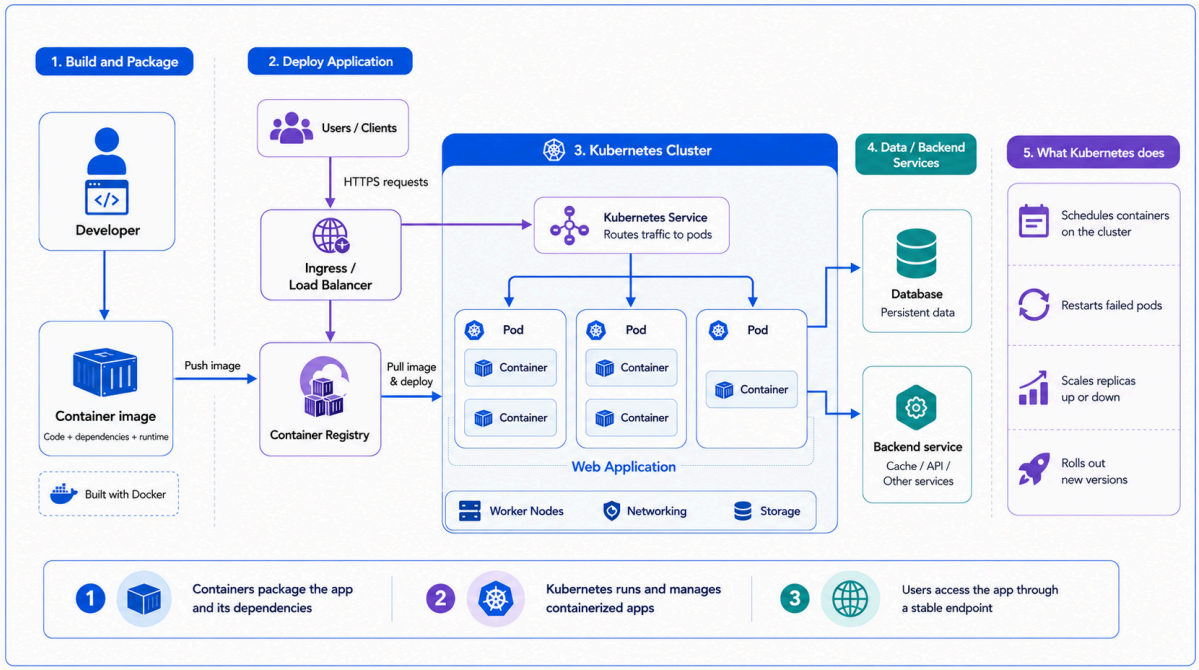
Calling an Azure MCP Server - Primer



In this chapter, we are looking at how a local Python application can call an Azure MCP Server, and why container hosting is useful for this kind of setup.

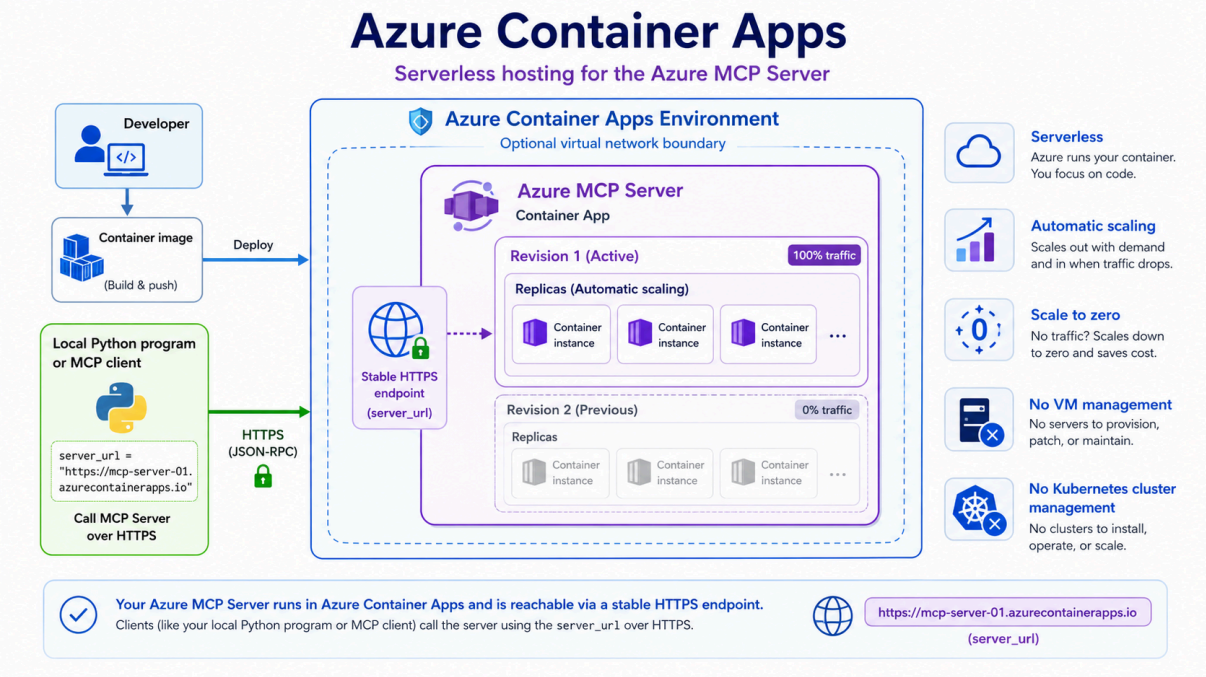
We have a Python application running locally on the developer's machine. This application acts as the MCP client. Instead of directly connecting to Azure Storage or calling many different Azure APIs itself, it sends an MCP request to an Azure MCP Server over HTTPS. The Azure MCP Server is hosted in Azure Container Apps. The server exposes MCP tools, such as tools for working with storage accounts, containers, blobs, or files. When the local Python app invokes a tool, the request goes to the MCP server. The server then accesses the required Azure resource, performs the operation, and returns the tool response back to the Python application.

So the important point is this: the local app does not need to contain all of the backend logic. The MCP server becomes the bridge between the client and Azure resources.



A container packages an application together with its dependencies and runtime. This makes the application easier to run consistently across different environments. For example, the same container image can run on a developer machine, in a test environment, or in the cloud.

In a Kubernetes setup, containers run inside pods. Kubernetes manages where those pods run, restarts failed pods, scales replicas up or down, and routes traffic through services and ingress. This is very powerful, but it also introduces operational complexity. Someone has to manage the cluster, networking, scaling rules, upgrades, and the overall Kubernetes environment.



That is where Azure Container Apps becomes useful.
We still deploy our MCP server as a container, but we do not have to manage a Kubernetes cluster directly. Azure Container Apps gives us a stable HTTPS endpoint, automatic scaling, revision management, and the ability to scale down when there is no traffic.
For our MCP server scenario, this is a very practical choice. We can build the MCP server, package it as a container image, deploy it to Azure Container Apps, and then call it from a local Python program, an AI agent, or another client using the server URL.
The key takeaway is that containers make the MCP server portable, Kubernetes explains the broader container orchestration model, and Azure Container Apps gives us a simpler managed way to run the server in Azure. This lets us focus on building MCP tools and connecting agents to real Azure resources, instead of spending time managing servers or Kubernetes clusters.